DBMS COLLEGE PROJECT - PROJECT REPORT

# Private Network Monitoring System

A web application that records which devices on an authorized network reached which destinations, and when. It stores that metadata in a normalized MySQL database and lets administrators search, investigate and report on it through a dashboard.

Python + FastAPI | SQLAlchemy 2 | MySQL 9 | React 19 + Vite | Tailwind CSS 4 | JWT + bcrypt

| | |
|---|---|
| **6** database tables | **32** REST API endpoints |
| **12** dashboard pages | **864** automated tests |

CONTENTS


1. Introduction
2. The big picture
3. The journey of one event
4. The collector
5. The backend (FastAPI)
6. Ingestion and the 30-second rule
7. The database
8. The frontend (React)
9. Security
10. Testing and quality
11. Running the project
12. Limitations and future work
13. Glossary


## 1 Introduction

### 1.1 The problem, in plain words

Every time a computer opens a website or connects to a server, it contacts another machine on the network. Someone who runs a network (a school computer lab, a library, a small office, a home) often needs to answer simple questions: *Which computer visited this site? What did this computer connect to yesterday afternoon? Which sites are most used?* Answering those questions from raw log files is slow and error prone.

This project solves that. A small program called a **collector** reports "device X reached destination Y at time T". A **backend** checks and stores those reports in a **database**. A **web dashboard** shows the stored data as tables, charts and reports, with search and filters.

## 1.2 What it records, and what it never records

> **Recorded (metadata only)**
>
> Which device, which destination IP address, the domain name when known, the time, and optionally the protocol (TCP/UDP) and the two port numbers.

> **Never recorded**
>
> Passwords, cookies, messages, page contents, request or response bodies, packet payloads, keystrokes, or the names of programs.

The system is intended **only for networks and devices the operator is authorized to monitor**, and whose users know about it. It contains no packet sniffing, scanning or discovery code.

## 1.3 Who uses it

| ROLE | WHAT THEY CAN DO |
| --- | --- |
| ADMIN | Everything an analyst can do, plus add, edit and disable nodes (observation points), and add, edit and disable user accounts. |
| ANALYST | Read-only: view the dashboard, nodes, devices, destinations and history, search and filter, and generate reports. |
| COLLECTOR | Not a person. A program that can only submit events, using its own API key that belongs to exactly one node. |

## 1.4 Project goals

- **Database design first:** a normalized schema with real foreign keys, constraints, indexes and transactions, explained in Section 7.
- **A working, full-stack system:** not only a user interface. Real data flows from a collector through an API into MySQL and back out to the browser.
- **Clean architecture:** each layer has one job (router, service, repository), so the code is easy to explain and test.
- **Security basics done properly:** hashed passwords, token authentication, role checks enforced on the server, and API keys for collectors.

# 2 The big picture

The system has four building blocks that pass data in one direction and answer questions in the other.

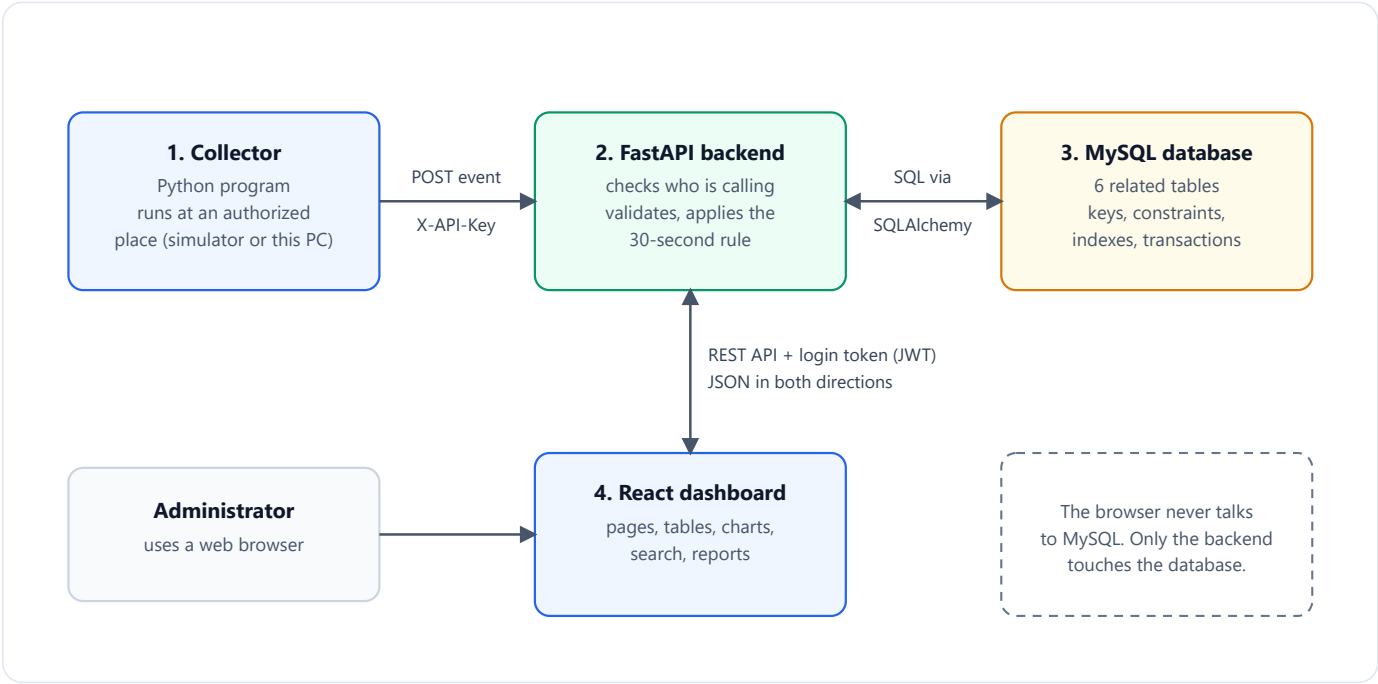



**Figure 1.** The four building blocks. Events travel left to right into the database. Questions from the administrator travel through the React dashboard and the FastAPI backend, and the answers come back as JSON.

### Why a separate collector?

A website cannot see your network by itself. Something has to sit at a place that can observe activity and report it. Keeping this as its own small program means the website and database do not change if the data source changes (simulator today, a real data source tomorrow).

### Why a backend between the browser and database?

The browser is not trusted. The backend is where logins, roles, validation and business rules are enforced, so nobody can bypass them by editing the web page. The frontend is only a convenience layer.

## 2.1 Folder structure

```
network-monitor/
  backend/        FastAPI application (api, services, repositories, models, schemas, core)
    alembic/      database migrations (for later schema changes)
    tests/        backend tests (run against real MySQL)
  collector/      collector.py (simulator + real local modes), api_client, tcp_collector, tests
  frontend/       React app (pages, components, hooks, services) + Vitest tests
  database/       schema.sql, seed.sql, queries.sql, reset.sql
  docs/           API notes, architecture notes, ER diagram, this report
```

# 3 The journey of one event

The best way to understand the whole system is to follow one event from start to finish. Imagine the computer `PC-LAB-A-001` opens `github.com`.

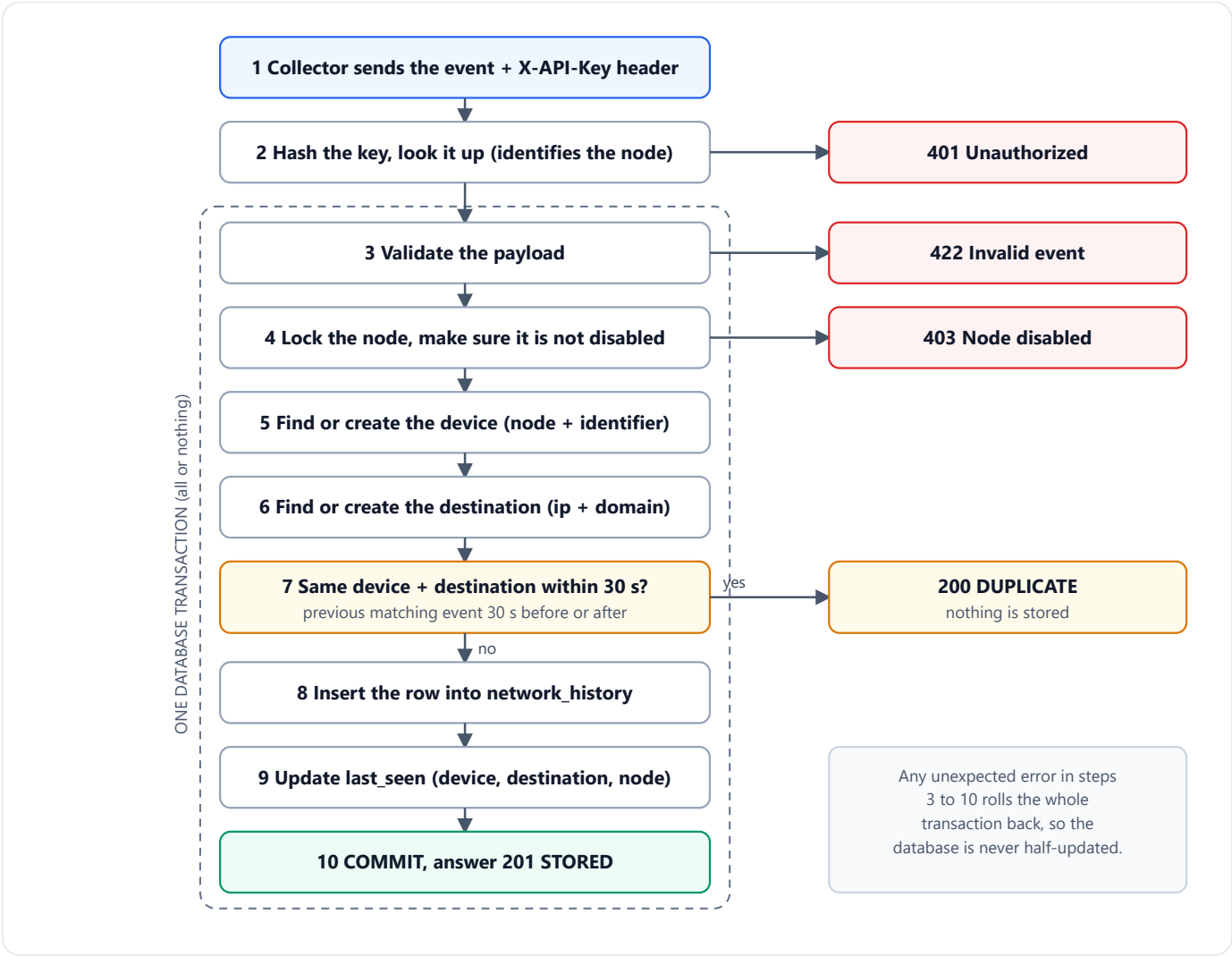



**Figure 2.** What the backend does for every event. The left column is the success path; the boxes on the right are the ways it can end early. The node is always taken from the API key, never from the event itself.

## 3.1 The same journey as a story

1. **The collector** notices that `PC-LAB-A-001` connected to `203.0.113.20` ( `github.com` ) and builds a small JSON message.
2. **It sends the message** to `POST /api/collector/events` with a secret API key in a header.
3. **The backend identifies the caller.** It hashes the key (SHA-256) and finds the matching row in `collector_api_keys`. That row says which node (for example *Computer Lab A*) the caller belongs to. If the key is unknown or revoked, the answer is 401.
4. **It validates the message.** A `nodeId` in the message is rejected on purpose: a collector must not be able to claim it belongs to a different node.
5. **It resolves the entities.** The device is looked up by node + `device_identifier` and created if new. The destination is looked up by IP + domain and created if new.
6. **It applies the 30-second rule** (Section 6). If the same device already reached the same destination within 30 seconds, nothing is stored and the answer is `DUPLICATE`.
7. **It stores the event** in `network_history`, updates the three `last_seen` columns and commits the transaction.
8. **The dashboard** asks the API for summaries and lists every few seconds, and the new row appears in the tables and charts.

# 4 The collector

The collector is one Python program ( `collector/collector.py` ) with several modes. All modes send events through the **same** API endpoint, so the backend neither knows nor cares which mode produced an event.

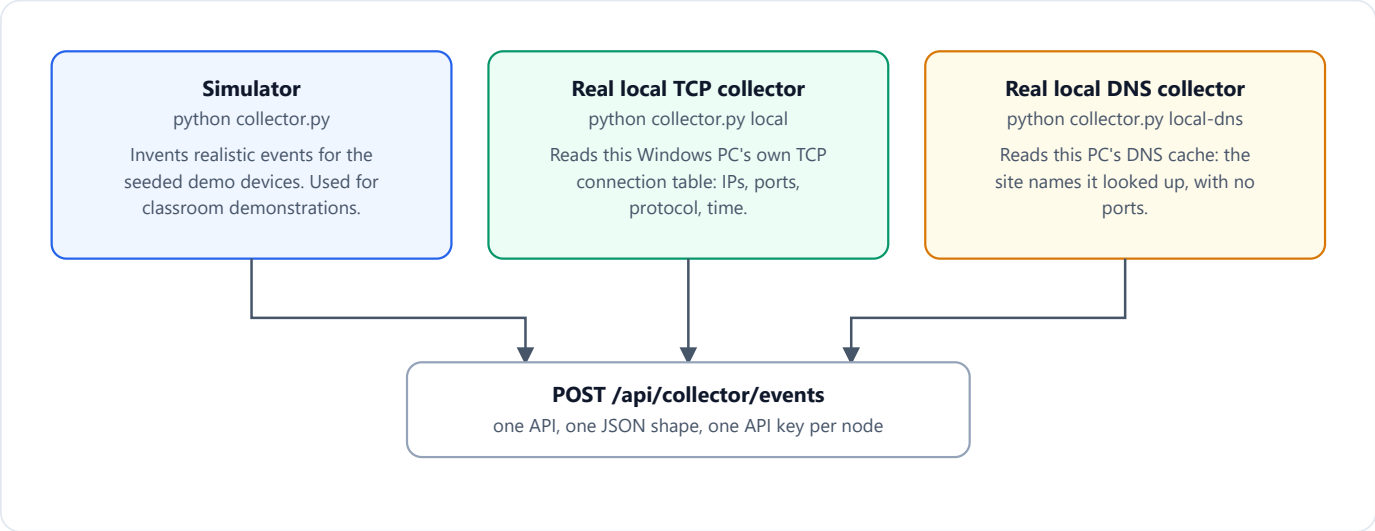



**Figure 3.** Three ways to produce events, one way to deliver them.

## 4.1 What an event looks like

```
{
  "deviceIdentifier": "PC-LAB-A-001",
  "deviceIp":         "192.168.10.21",
  "destinationIp":    "203.0.113.20",
  "domain":           "github.com",
  "timestamp":        "2026-09-21T11:08:30",
  "protocol":         "TCP",        // optional
  "sourcePort":       56188,        // optional
  "destinationPort": 443            // optional
}
```

There is deliberately **no node id**. The node comes from the API key. Protocol and ports are optional because the simulator and the DNS mode cannot observe them.

## 4.2 The simulator

Builds events for the seeded demo devices using documentation IP ranges ( `192.0.2.0/24` , `198.51.100.0/24` , `203.0.113.0/24` ) that do not belong to real organizations. It deliberately repeats some events so the 30-second rule can be shown live ( `python collector.py demo-dedup` ). The dashboard marks demonstration data with a "Demo data" badge.

## 4.3 The real local TCP collector

On Windows, the operating system keeps a table of every TCP connection. The collector reads it with the built-in IP Helper function `GetExtendedTcpTable` through Python's standard `ctypes` . That needs **no administrator rights and no extra software** (no Npcap, no Scapy).

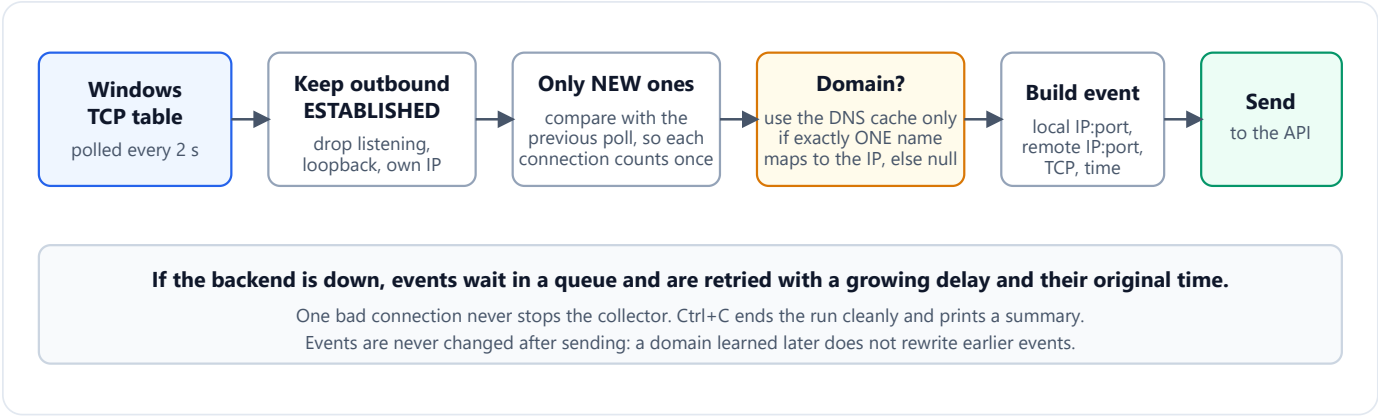



**Figure 4.** One look at the connection table, from raw Windows data to a stored event.

> **Important limits**
>
> **This collector sees only the computer it runs on.** It does not monitor other laptops, phones or users on the Wi-Fi, or traffic through a router. It is **TCP only**: UDP, including the QUIC protocol many browsers use for sites like YouTube, has no connection table, so those visits can be missing. It polls a snapshot, so very short connections can be missed. It provides *local TCP connection visibility*, not a complete browsing history.

## 4.4 How the domain name is chosen

A connection table contains only IP addresses. To attach a name, the collector looks in the Windows DNS cache and uses a name **only when exactly one cached name maps to that IP address**. Large services (CDNs, cloud front ends) share IP addresses between thousands of names, so a single name cannot be chosen honestly; the domain is then left empty ( `null` ) rather than guessed. The collector never performs reverse DNS lookups.

# 5 The backend (FastAPI)

The backend is a REST API written in Python. It receives HTTP requests, enforces who is allowed to do what, applies the business rules, and reads and writes MySQL.

## 5.1 Layered design

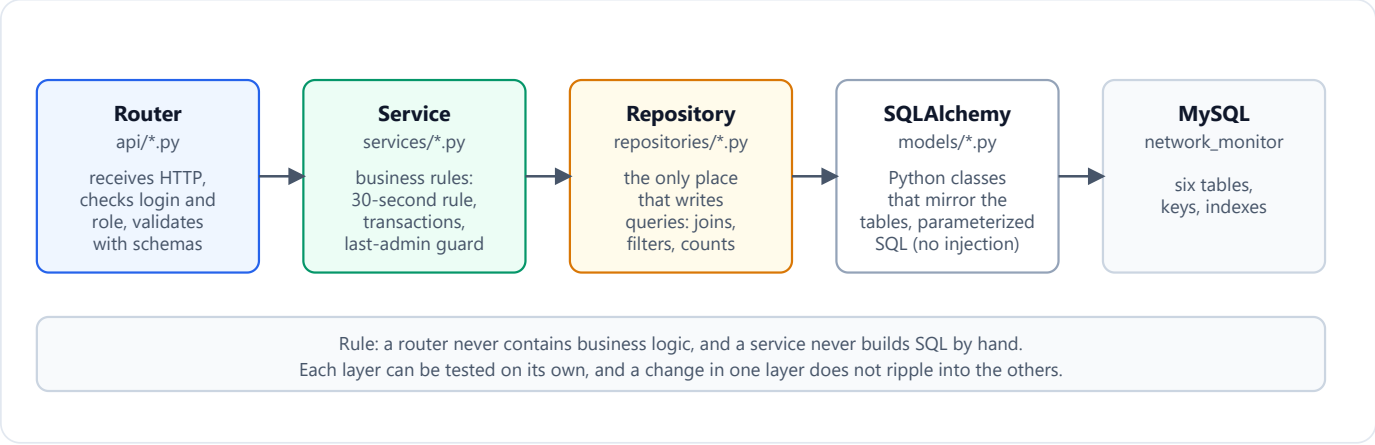



**Figure 5.** Every request passes through the same four layers before reaching the database.

## 5.2 Folder map of `backend/app`

| FOLDER | CONTENTS | PURPOSE |
|---|---|---|
| `api/` | auth, nodes, devices, destinations, history, dashboard, reports, users, collector, health | HTTP endpoints (routers) |
| `services/` | auth, collector (ingestion), activity, node, user, report, dashboard ... | Business rules |
| `repositories/` | one per table plus stats and filters | All database queries |
| `models/` | User, Node, Device, Destination, CollectorApiKey, NetworkHistory | ORM classes mirroring the schema |
| `schemas/` | request and response shapes (Pydantic) | Validation and API contracts |
| `core/` | config, security, dependencies, query parameters | Settings, hashing, tokens, auth checks, pagination |
| `exceptions/` | error classes and handlers | Consistent, safe error responses |
| `db/` | engine and session | The MySQL connection |

## 5.3 Two ways to prove who you are

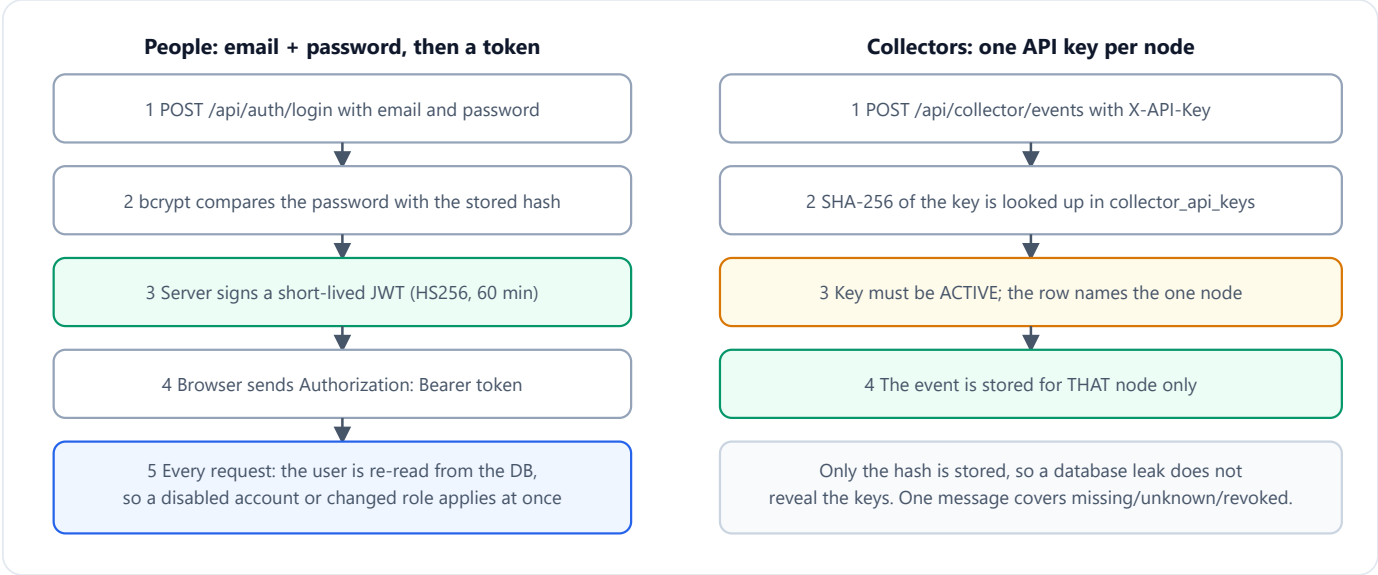



**Figure 6.** Two separate authentication paths. Roles (ADMIN, ANALYST) are checked on the server for every protected route; hiding a button in the browser is never the only protection.

## 5.4 Validation and error handling

- **Pydantic schemas** check every incoming value (types, lengths, IP address format, port range 1 to 65535, allowed protocol values). Unknown fields are rejected.
- **Pagination is done by the database** ( `page` , `pageSize` up to 100), so the browser never receives thousands of rows at once.
- **HTTP status codes** are meaningful: 200, 201, 204, 400, 401, 403, 404, 409, 422, 503. A 409 means a conflict such as a duplicate node code or an attempt to disable the last administrator.
- **Errors never leak internals.** The server logs details but returns generic messages, and it does not echo rejected input back (it could be a password).

## 5.5 The 32 API endpoints

| AREA | ENDPOINTS | WHO |
|---|---|---|
| Health | `GET /api/health · GET /api/health/db` | public · signed in |
| Auth | `POST /api/auth/login · POST /api/auth/logout · GET /api/auth/me` | public (login) · signed in |
| Dashboard | `GET /api/dashboard/summary · /activity · /top-destinations · /top-devices` | signed in |
| Nodes | `GET /api/nodes · GET /api/nodes/{id} · POST /api/nodes · PUT /api/nodes/{id} · PATCH /api/nodes/{id}/status` | read: signed in · write: ADMIN |
| Devices | `GET /api/devices · /{id} · /{id}/history` | signed in |
| Destinations | `GET /api/destinations · /{id} · /{id}/history` | signed in |
| History | `GET /api/history · GET /api/history/{id}` | signed in |
| Reports | `GET /api/reports/top-destinations · /top-devices · /activity-by-node · /activity-over-time` | signed in |
| Users | `GET /api/users · GET /{id} · POST · PUT /{id} · PATCH /{id}/status` | ADMIN |
| Collector | `POST /api/collector/events` | API key |

Interactive documentation for all of these is generated automatically by FastAPI at `/docs` (it can be switched off for a real deployment with `ENABLE_API_DOCS=false`).

# 6 Ingestion and the 30-second rule

A browser loading one page can open many connections to the same server within a few seconds. Storing every one would fill the history with noise. The project's special rule fixes that:

> **The 30-second rule**
>
> **Same device + same destination + another event within 30 seconds = duplicate, and a duplicate is not stored.** After 30 seconds the same device reaching the same destination is stored again, because that is a new visit.

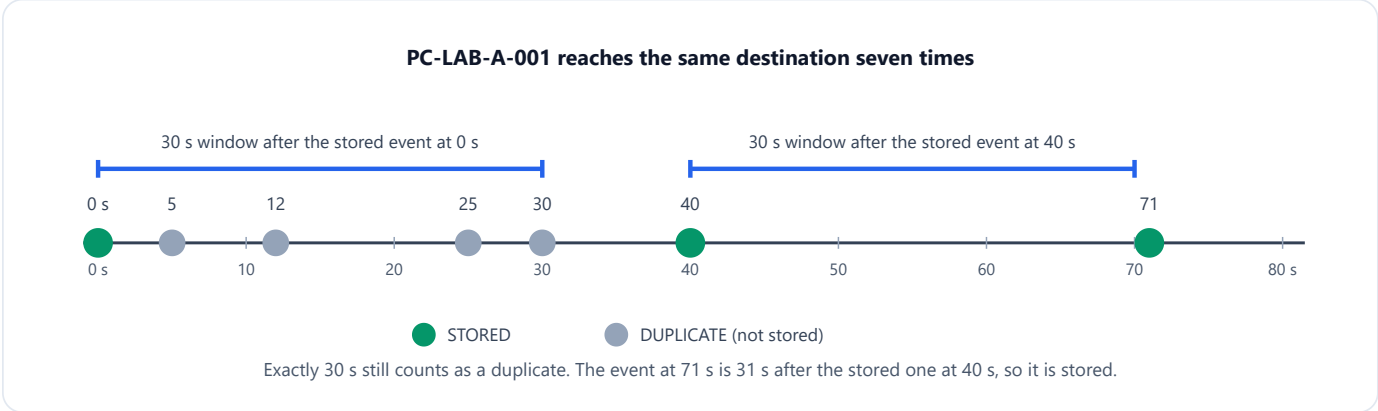



**Figure 7.** The classroom demonstration (`python collector.py demo-dedup`). Only the green events (0 s, 40 s, 71 s) reach the database.

## 6.1 Why it lives in the backend, not the database

- It cannot be a `UNIQUE(device_id, destination_id)` constraint: that would forbid the same device ever reaching the same destination twice, even a month later.
- It cannot live in the frontend or collector alone: any client could skip it. The backend is the single place all events pass through, so the rule is enforced once, for everyone.
- Ports and protocol are **not** part of the rule. Two connections from one device to one server within 30 seconds are one event, whatever their ports are.

## 6.2 Making it safe when events arrive at the same time

Two events from the same device could arrive at the same moment and both pass the duplicate check. To prevent that, the service reads with `SELECT ... FOR UPDATE` (a *locking read*) and always takes the locks in the same order: **node, then device, then destination**. A fixed order means two transactions can never wait on each other in a circle (a deadlock). If MySQL still reports a lock timeout, the API answers 503 and the collector simply resends, which is safe because duplicates are detected.

The window is checked in both directions (30 seconds before and after), so an event that arrives late and out of order is also handled correctly.

# 7 The database

The database is the heart of the project. It is a MySQL 8+ (running MySQL 9.7) database named `network_monitor` with **exactly six application tables**, all InnoDB, character set `utf8mb4`, with `BIGINT UNSIGNED AUTO_INCREMENT` primary keys and `DATETIME` timestamps.

## 7.1 Why MySQL and a relational design

- The data is naturally **relational**: devices belong to nodes, events refer to devices and destinations. Foreign keys let the database itself guarantee those links are valid.
- Questions such as "top destinations", "devices per node" or "activity per day" are joins and aggregates, which SQL does very well.
- **Transactions** (all-or-nothing) are needed when one event updates four tables at once.
- The schema is created from a plain SQL file (`schema.sql`), so it is visible and explainable; the Python models mirror it, rather than generating it.

## 7.2 The entity-relationship diagram

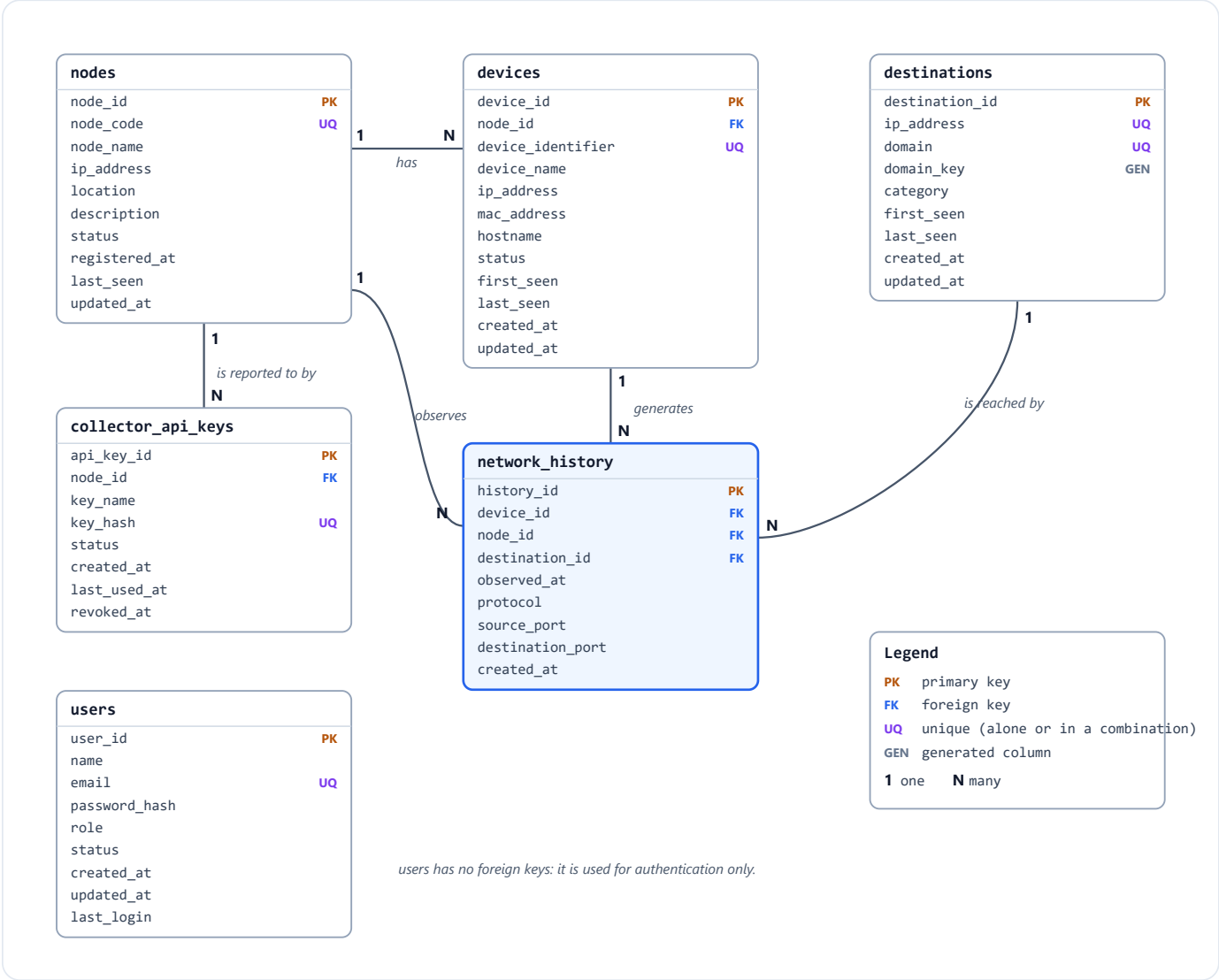



**Figure 8.** The six tables and how they relate. `network_history` (highlighted) is the central table: each row ties one device, one node and one destination to one moment in time.

| PARENT | CHILD | TYPE | MEANING |
|---|---|---|---|
| nodes | devices | 1 : N | A node has many devices; a device belongs to exactly one node. |
| nodes | collector_api_keys | 1 : N | A node can have several keys (for example while rotating); a key belongs to one node. |
| nodes | network_history | 1 : N | Every event records which observation point saw it. |
| devices | network_history | 1 : N | A device generates many events. |
| destinations | network_history | 1 : N | A destination is reached by many events (from many devices). |
| users | no relationships: accounts are used for sign-in and permissions only. | | |

## 7.3 Table by table

For each table: what it stores, why it exists, how the application uses it, and the design decisions behind it.

### 1. `users` - who may sign in

**Stores** the accounts of the people who use the dashboard.

| COLUMN | TYPE | NOTES |
|---|---|---|
| user_id | BIGINT UNSIGNED | Primary key, auto-increment |
| name | VARCHAR(100) | Display name |
| email | VARCHAR(255) | **UNIQUE**; the login name; stored lower-case |
| password_hash | VARCHAR(255) | bcrypt hash, never the password itself |
| role | ENUM('ADMIN','ANALYST') | Default ANALYST; decides what the account may do |
| status | ENUM('ACTIVE','DISABLED') | Disabled accounts cannot sign in |
| created_at, updated_at, last_login | DATETIME | `updated_at` refreshes automatically; `last_login` is NULL until the first login |

**How it is used.** At login the backend finds the row by `email` (the unique index makes this one lookup), compares the password with `password_hash` using bcrypt, and records `last_login`. On every later request the user row is read again to check `status` and `role`, so disabling an account or changing a role takes effect immediately. The Users page (admin only) creates, edits and disables rows.

**Why it looks like this.** Users are people, not network data, so there is no foreign key to the monitoring tables. Accounts are *disabled, not deleted*, so history of who did what is not lost, and the application refuses to disable or demote the **last active administrator** so nobody can lock everyone out. The `ENUM` columns let MySQL reject any role or status that is not on the list.

### 2. `nodes` - authorized observation points

**Stores** each place where monitoring is authorized: a computer lab, the library, an office, or "this PC".

| COLUMN | TYPE | NOTES |
|---|---|---|
| `node_id` | `BIGINT UNSIGNED` | Primary key |
| `node_code` | `VARCHAR(50)` | **UNIQUE** short code such as `LAB-A` |
| `node_name, location` | `VARCHAR(100 / 150)` | Human-readable name and place |
| `ip_address` | `VARCHAR(45)` | Address of the observation point; 45 characters fit the longest IPv6 text form |
| `description` | `VARCHAR(500) NULL` | Optional |
| `status` | `ENUM('ACTIVE','INACTIVE','OFFLINE')` | Index `idx_nodes_status` |
| `registered_at, last_seen, updated_at` | `DATETIME` | `last_seen` indexed; NULL until the node reports |

**How it is used.** The Nodes page (admin) adds, edits and disables nodes. When an event arrives, the node found through the API key is **locked**; if its status is `INACTIVE` the event is refused with 403, and otherwise its `last_seen` moves forward. The dashboard's "Active nodes" figure counts rows with status ACTIVE, and the reports group activity by node.

**Why it looks like this.** A node is the *parent* of devices and keys, which gives every event a clear origin. Nodes are never deleted (history refers to them); they are set to `INACTIVE` instead, which also acts as an off switch for a collector.

### 3. `devices` - the machines being observed

**Stores** one row per logical device seen by a node.

| COLUMN | TYPE | NOTES |
|---|---|---|
| `device_id` | `BIGINT UNSIGNED` | Primary key |
| `node_id` | `BIGINT UNSIGNED` | **FK** to `nodes`, ON DELETE RESTRICT, ON UPDATE CASCADE |
| `device_identifier` | `VARCHAR(100)` | The **logical identity** (for example `PC-LAB-A-001`), unique within a node |
| `device_name, hostname, mac_address` | `VARCHAR NULL` | Optional descriptive details; shown as a dash when unknown |
| `ip_address` | `VARCHAR(45)` | Current address; can change; indexed |
| `status` | `ENUM('ACTIVE','INACTIVE','UNKNOWN')` | Default UNKNOWN |
| `first_seen, last_seen, created_at, updated_at` | `DATETIME` | `last_seen` is indexed |

**Constraints.** `UNIQUE (node_id, device_identifier)` means a device name cannot repeat inside one node. `UNIQUE (node_id, device_id)` looks redundant (device_id is already unique) but is required: it is the target of the composite foreign key from `network_history` (Section 7.4).

**How it is used.** Devices are created *automatically* by the first event that mentions them: the service looks for `node_id + device_identifier` and inserts a row if none exists. Later events update `last_seen` and the current IP. The Devices page lists them with search (by name, IP, hostname) and filters (node, status); the device detail page shows the history of that one device.

**Why it looks like this.** The identity is the *identifier*, not the IP address, because IPs change (DHCP). If a laptop gets a new address, it stays the same device with one continuous history rather than becoming a stranger.

### 4. `destinations` - where traffic went

**Stores** each distinct place that was reached, once, no matter how many devices or events refer to it.

| COLUMN | TYPE | NOTES |
|---|---|---|
| `destination_id` | `BIGINT UNSIGNED` | Primary key |
| `ip_address` | `VARCHAR(45)` | Destination IP (IPv4 or IPv6) |
| `domain` | `VARCHAR(255) NULL` | Name when known; NULL when no name was observed; indexed |
| `domain_key` | `VARCHAR(255) GENERATED` | `COALESCE(domain, '')`, STORED. Internal helper, never shown to users |
| `category` | `ENUM(SEARCH, SOCIAL, DEVELOPMENT, EDUCATION, VIDEO, CLOUD, OTHER, UNKNOWN)` | Descriptive label; default UNKNOWN; indexed |
| `first_seen, last_seen, created_at, updated_at` | `DATETIME` | `last_seen` indexed |

**The NULL problem and the generated column.** The identity of a destination is `ip_address + domain`, and `domain` may be NULL. In MySQL a UNIQUE index allows *many* rows with NULL, so `UNIQUE(ip_address, domain)` would allow `(203.0.113.50, NULL)` to be inserted again and again. The generated column turns NULL into an empty string, and `UNIQUE (ip_address, domain_key)` then works for every case while `domain` itself stays truly NULL.

**How it is used.** On each event the service looks up `ip_address` + `domain_key` and reuses the row or creates it. It powers the Destinations page, the destination investigation page (total events, unique devices, nodes that saw it, devices that accessed it), the "top destinations" chart and the category breakdown.

**Why it looks like this.** Storing each destination once avoids repeating the same name and IP in millions of history rows. The `category` is application metadata for readability and is explicitly *not* a security verdict.

### 5. `collector_api_keys` - how collectors prove who they are

**Stores** the credentials that collectors use, one or more per node.

| COLUMN | TYPE | NOTES |
|---|---|---|
| `api_key_id` | `BIGINT UNSIGNED` | Primary key |
| `node_id` | `BIGINT UNSIGNED` | **FK** to `nodes`, RESTRICT; the node this key speaks for |
| `key_name` | `VARCHAR(100)` | A label such as "local collector" |
| `key_hash` | `VARCHAR(255)` | **UNIQUE**; SHA-256 of the key. The key itself is never stored |
| `status` | `ENUM('ACTIVE','REVOKED')` | Revoking = changing this value |
| `created_at, last_used_at, revoked_at` | `DATETIME` | `last_used_at` is updated on each authenticated use |

**How it is used.** For each event the backend hashes the `X-API-Key` header and finds the row through the unique index on `key_hash`. If the row exists and is ACTIVE, `node_id` tells the backend which node the event belongs to. A helper script (`backend/create_collector_key.py`) creates a node and key and prints the key **once**.

**Why it looks like this.** A separate table lets a node have several keys (rotation) and lets a key be revoked without deleting anything. Hashing means a leaked database does not leak working keys. A fast SHA-256 is appropriate here (unlike for passwords) because keys are long random secrets, and a fast, repeatable hash is what allows a lookup by hash. The key is bound to the node, which is why a collector can never write events for another node.

### 6. `network_history` - the events (the central table)

**Stores** one row per accepted event: *this device, seen by this node, reached this destination, at this time*.

| COLUMN | TYPE | NOTES |
|---|---|---|
| `history_id` | `BIGINT UNSIGNED` | Primary key |
| `device_id` | `BIGINT UNSIGNED` | **FK** to `devices` |
| `node_id` | `BIGINT UNSIGNED` | **FK** to `nodes` |
| `destination_id` | `BIGINT UNSIGNED` | **FK** to `destinations` |
| `observed_at` | `DATETIME NOT NULL` | When the collector saw it |
| `protocol` | `ENUM('TCP','UDP') NULL` | Optional connection detail |
| `source_port, destination_port` | `SMALLINT UNSIGNED NULL` | Optional; 0 to 65535 fits in two bytes |
| `created_at` | `DATETIME` | When the row was inserted |

**How it is used.** Everything the dashboard shows is computed from this table: the recent-activity table, the activity charts, the history search page, device and destination histories, node activity and all four reports. The duplicate check queries it by device and destination within a time window.

**Why it looks like this.** It is a "tall and narrow" table of ids and a time, so adding millions of rows stays cheap. It deliberately stores **no names or addresses**: those are reached through joins (Section 7.6). Protocol and ports describe *this one connection* (every connection has its own ephemeral source port), so they belong on the event row, not on devices or destinations. They are nullable because not every collector can observe them; older rows simply show a dash.

## 7.4 Foreign keys, and the composite key that prevents impossible events

`network_history` has four foreign keys, all `ON UPDATE CASCADE ON DELETE RESTRICT`:

1. `device_id → devices(device_id)`
2. `node_id → nodes(node_id)`
3. `destination_id → destinations(destination_id)`
4. **`(node_id, device_id) → devices(node_id, device_id)`** (composite)

The first three only prove each id exists. They would still allow a contradiction: an event saying *device 7 was seen by node 2* when device 7 actually belongs to node 1. The composite key closes that gap, because MySQL must find a `devices` row with **that exact node and device together**. This is why `devices` carries the extra `UNIQUE (node_id, device_id)`: a foreign key needs a unique target.

| ATTEMPTED EVENT | RESULT |
|---|---|
| `device_id = 7, node_id = 1 (device 7 belongs to node 1)` | `accepted` |
| `device_id = 7, node_id = 2 (device 7 does not belong to node 2)` | `rejected by MySQL: foreign key violation` |

The application also checks this, but the database enforces it even if some future code forgets to.

## 7.5 Indexes: what each one is for

An index is a sorted lookup structure that lets MySQL jump to rows instead of reading the whole table. Indexes make reads faster but writes slightly slower, so only indexes that serve a real query were added.

| INDEX | TABLE | QUERY IT SPEEDS UP |
|---|---|---|
| `(device_id, observed_at)` | `network_history` | A device's history, newest first; the 30-second duplicate check |
| `(node_id, observed_at)` | `network_history` | Node detail page, activity by node |
| `(destination_id, observed_at)` | `network_history` | Destination investigation, "who reached this" |
| `(observed_at)` | `network_history` | Dashboard counts, date filters, reports over time |
| `UNIQUE (node_id, device_identifier)` | `devices` | Finding the device for each incoming event |
| `UNIQUE (ip_address, domain_key)` | `destinations` | Finding the destination for each incoming event |
| `UNIQUE (key_hash)` | `collector_api_keys` | Authenticating a collector in one lookup |
| `UNIQUE (email)` | `users` | Login lookup |
| `ip_address, status, last_seen` | `devices` | Search by IP, filter by status, sort by recent |
| `domain, category, last_seen` | `destinations` | Search by domain, filter by category |
| `status, last_seen` | `nodes` | Counting active nodes, finding silent ones |

## 7.6 Normalization: why the history table holds only ids

The schema is designed to about **Third Normal Form (3NF)**: every fact is stored in exactly one place.

- **1NF:** every column holds a single value; there are no lists inside a cell.
- **2NF:** every table has a single-column primary key, so no column can depend on only part of a key.
- **3NF:** a column depends on the key and nothing but the key. For example the node's name is not copied into `devices` or `network_history`, because it depends on the node, not on the device or the event.

**What it looks like for one event.** Suppose `PC-LAB-A-001` reaches `github.com`. The event itself is one small row; the descriptive details live in their own tables and are joined when needed (example values):

| NETWORK_HISTORY | | | | | | | |
|---|---|---|---|---|---|---|---|
| HISTORY_ID | DEVICE_ID | NODE_ID | DESTINATION_ID | OBSERVED_AT | PROTOCOL | SOURCE_PORT | DESTINATION_PORT |
| 1749 | 3 | 1 | 2 | 2026-09-21 20:18:57 | TCP | 53011 | 443 |

| DEVICES (ID 3) | |
|---|---|
| device_identifier | PC-LAB-A-001 |
| ip_address | 192.168.10.21 |
| node_id | 1 |

| NODES (ID 1) | |
|---|---|
| node_code | LAB-A |
| node_name | Computer Lab A |
| status | ACTIVE |

| DESTINATIONS (ID 2) | |
|---|---|
| ip_address | 203.0.113.20 |
| domain | github.com |
| category | DEVELOPMENT |

If the lab is renamed, one `UPDATE` on `nodes` changes every past and future event's display, instead of millions of rows.

> **A deliberate exception**
>
> `last_seen` on nodes, devices and destinations is technically derivable from `MAX(observed_at)` in the history. It is stored anyway (and updated in the same transaction as the event) so that lists such as "devices, most recently active first" do not need an expensive aggregate over the whole history. This is a conscious, documented trade-off of a little redundancy for speed.

## 7.7 Deletion policy: history is never lost by accident

Every foreign key uses `ON DELETE RESTRICT`. MySQL will refuse to delete a node that still has devices, keys or history, and refuse to delete a device or destination that events refer to. There is no *cascade* that could silently wipe history. Instead of deleting, the application uses statuses ( `ACTIVE` , `INACTIVE` , `OFFLINE` , `DISABLED` , `REVOKED` ). Reports and investigations therefore always have the full record.

## 7.8 Transactions and integrity

Storing one event changes several rows: the new history row, the `last_seen` of the device, destination and node, the key's `last_used_at` , and possibly a new device or destination row. They are performed in **one transaction**: either every change is committed together or, on any error, none is. This is the **A**tomicity and **C**onsistency of ACID in practice. The locking reads give **I**solation between simultaneous events, and InnoDB's log gives **D**urability.

| MECHANISM | WHAT IT GUARANTEES |
|---|---|
| Primary keys | Every row is uniquely identifiable |
| Foreign keys (incl. composite) | No event can point at something that does not exist, or at a device from the wrong node |
| UNIQUE constraints | No duplicate emails, node codes, device identifiers per node, destinations, or key hashes |
| NOT NULL, ENUM | Required values exist, and statuses and roles come from a fixed list |
| Transactions + locking reads | All-or-nothing writes; no race between simultaneous events |
| ON DELETE RESTRICT | No accidental loss of history |

## 7.9 The SQL files, and how the database is created and changed

| FILE | PURPOSE | SAFE? |
|---|---|---|
| `database/schema.sql` | Creates the database (if missing) and the six tables. Run once on a new setup. | Never drops or truncates anything |
| `database/seed.sql` | Demo data: 2 users, 4 nodes, 17 devices, 12 destinations, 4 collector keys and 815 events over the last 7 days, including cases that show the 30-second rule. Demo passwords are stored as bcrypt hashes. | No destructive commands |
| `database/queries.sql` | 22 read-only demonstration queries (joins, GROUP BY, HAVING, LEFT JOIN, date filters, audits). | SELECT only |
| `database/reset.sql` | Drops and recreates the database for an intentional local reset. | **The only destructive file.** Never run automatically by the app or by tools |
| `backend/alembic/` | Controlled migrations for *later* schema changes. Migration `0001` added `protocol`, `source_port` and `destination_port` to the live database without touching existing rows. It is idempotent (does nothing if the columns already exist, as on a fresh install). | Additive; creates the helper table `alembic_version` |

The application never creates, drops or resets tables by itself. It connects to the existing database, and a test checks that the Python models match the live schema.

## 7.10 Example queries (from `queries.sql`)

### RECENT ACTIVITY: A FOUR-TABLE JOIN

```
SELECT h.observed_at, dv.device_identifier, n.node_name,
       d.ip_address AS destination_ip, d.domain
FROM network_history h
JOIN devices dv      ON dv.device_id      = h.device_id
JOIN nodes n         ON n.node_id         = h.node_id
JOIN destinations d  ON d.destination_id  = h.destination_id
ORDER BY h.observed_at DESC
LIMIT 20;
```

This one query reassembles names from ids. It is what the dashboard's "Recent network activity" table uses (with a `LIMIT`).

### MOST VISITED DESTINATIONS: GROUP BY AND HAVING

```
SELECT d.domain, COUNT(*) AS visits, COUNT(DISTINCT h.device_id) AS unique_devices
FROM network_history h
JOIN destinations d ON d.destination_id = h.destination_id
GROUP BY d.destination_id, d.domain
HAVING COUNT(*) >= 50          -- only busy destinations
ORDER BY visits DESC;
```

### DEVICES THAT NEVER PRODUCED AN EVENT: LEFT JOIN ANTI-JOIN

```
SELECT dv.device_identifier
FROM devices dv
LEFT JOIN network_history h ON h.device_id = dv.device_id
WHERE h.history_id IS NULL;
```

### THE DUPLICATE CHECK THE BACKEND RUNS

```
SELECT history_id, observed_at
FROM network_history
WHERE device_id = ? AND destination_id = ?
  AND observed_at BETWEEN (? - INTERVAL 30 SECOND) AND (? + INTERVAL 30 SECOND)
ORDER BY observed_at DESC
LIMIT 1
FOR UPDATE;
```

A row returned means "duplicate: do not insert". It is fast because of the index on `(device_id, observed_at)`.

# 8 The frontend (React)

The frontend is a single-page application. After login the browser loads the app once, then moves between pages without reloading, fetching only data from the API.

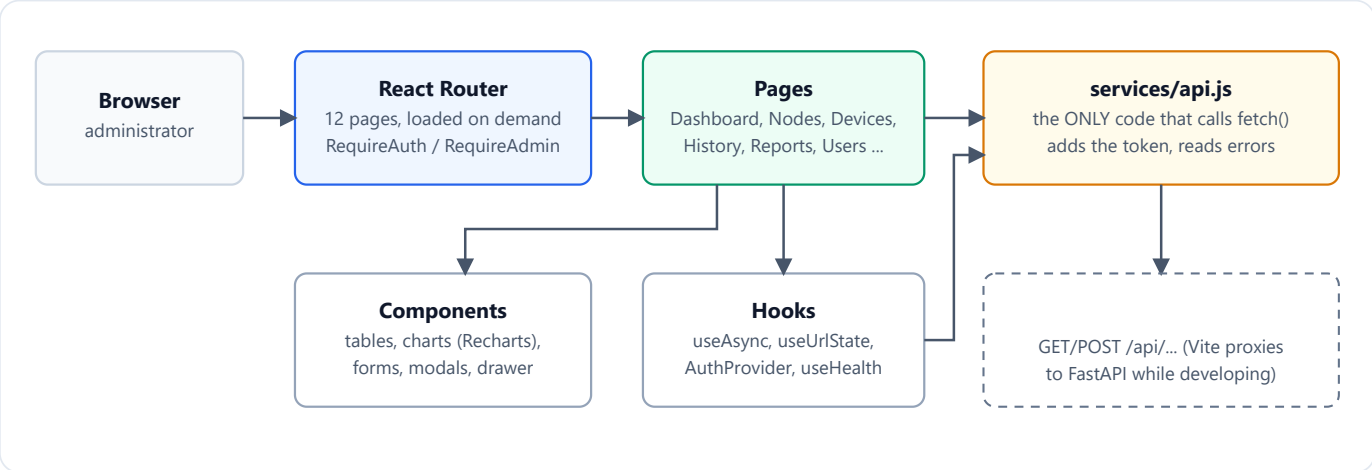



**Figure 9.** Pages use components for display and hooks for data. Hooks call one service layer that talks to the backend. No component ever calls the network directly.

## 8.1 What the dashboard looks like

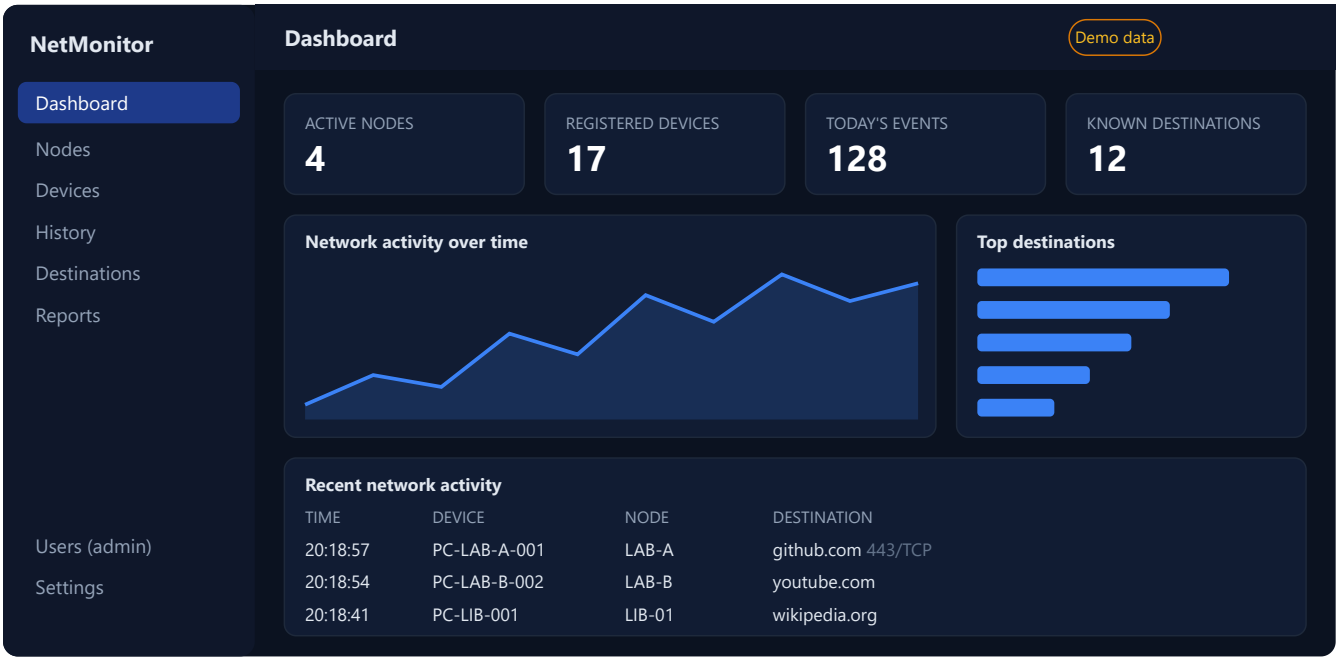



**Figure 10.** A simplified sketch of the dashboard (a dark "network operations" style). The real page also has "Most active devices", "Node activity", a time-range switch (24 hours, 7 days, 30 days), a Refresh button and live updates every 15 seconds.

## 8.2 Pages

| ROUTE | PAGE | WHAT IT DOES | WHO |
|---|---|---|---|
| `/login` | Sign in | Email and password; shows demo accounts in development builds only | everyone |
| `/dashboard` | Dashboard | Four key numbers, activity chart, top destinations, most active devices, node activity, recent events | signed in |
| `/nodes, /nodes/:id` | Nodes | List with search and status filter; detail with devices, 7-day chart and recent activity; add, edit, disable | read: all · write: ADMIN |
| `/devices, /devices/:id` | Devices | List with filters; detail with metadata, activity chart, recent destinations and a paged history | signed in |
| `/history` | Network history | The main investigation page: search text, node, destination and date filters, paging, event detail panel with protocol and ports | signed in |
| `/destinations, /destinations/:id` | Destinations | List and investigation: events, unique devices, nodes that saw it, devices that accessed it | signed in |
| `/reports` | Reports | Top destinations, top devices, node activity, activity over time, for 24 h / 7 d / 30 d or a custom range | signed in |
| `/users` | Users | Add, edit, disable accounts | ADMIN |
| `/settings` | Settings | Your account, system health, how the 30-second rule works, sign out | signed in |

## 8.3 Behaviors worth knowing

- **Every data page handles three states:** a loading skeleton, an empty state ("No network activity found. Try changing your filters.") and an error state with a Try again button. A missing record shows "Not found" with a link back.
- **Search and filters run on the backend.** The browser sends the filters in the request; it never downloads the whole table and filters it itself. The filters are kept in the address bar, so a filtered view can be bookmarked or shared.
- **The session** (token and user) is kept in the browser's local storage. A request answered with 401 ends the session and returns to the login page.
- **Role guards in the UI** hide pages and buttons the user cannot use, but the backend enforces the same rules, so they are a convenience only.
- **Responsive:** a persistent sidebar on desktop, a slide-in menu on phones; wide tables scroll inside their own box; the source-port column hides on narrow screens. Verified at 1280, 768 and 375 pixels wide.
- **Honest display:** a missing value is a dash, a destination without a name shows its IP marked "(unresolved)", and nothing is invented.

## 9 Security

| AREA | WHAT WAS DONE |
|---|---|
| Passwords | Hashed with bcrypt (work factor 12, never below 10). Never stored, returned or logged. Unknown emails take the same time as wrong passwords, and the message is the same for both. |
| Login tokens | JWT signed with HS256 (the algorithm is fixed, so a token cannot choose its own). The token carries only the user id and an expiry (60 minutes); role and status are re-read from the database on every request. The server refuses to start without a random secret of at least 32 characters, and rejects placeholder values. |
| Roles | ADMIN and ANALYST are enforced by the backend on every protected route (401 not signed in, 403 not allowed). The last active admin cannot be disabled or demoted. |
| Collector keys | Random secrets, stored only as SHA-256 hashes, tied to one node, revocable. The node always comes from the key, never from the payload. |
| SQL injection | All queries go through SQLAlchemy with bound parameters. Tests send injection strings through every search box, the login form and URL ids. SQL errors never echo bound values. |
| Input validation | Pydantic checks types, lengths, IP formats, port ranges and protocol values; unknown fields are rejected; page size and page number are capped. |
| Browser protection | CORS allows only the configured frontend origin, without credentials. Responses carry `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy` and `Cache-Control: no-store`. React escapes all displayed data, and no raw HTML is injected. |
| Errors and logging | No stack traces or internals reach the client; validation errors do not repeat the submitted input. |
| Exposure | Interactive API docs can be switched off for deployment. The database health check needs a login. Secrets live in environment variables and `.env` files that are not committed. |
| Privacy scope | Metadata only. The collectors contain no packet capture, scanning, discovery, payload or process-name collection, and the local collector sees only the computer it runs on. |
| Dependencies | Checked with `pip-audit` and `npm audit` during the final audit (no known vulnerabilities at that time). |

# 10 Testing and quality

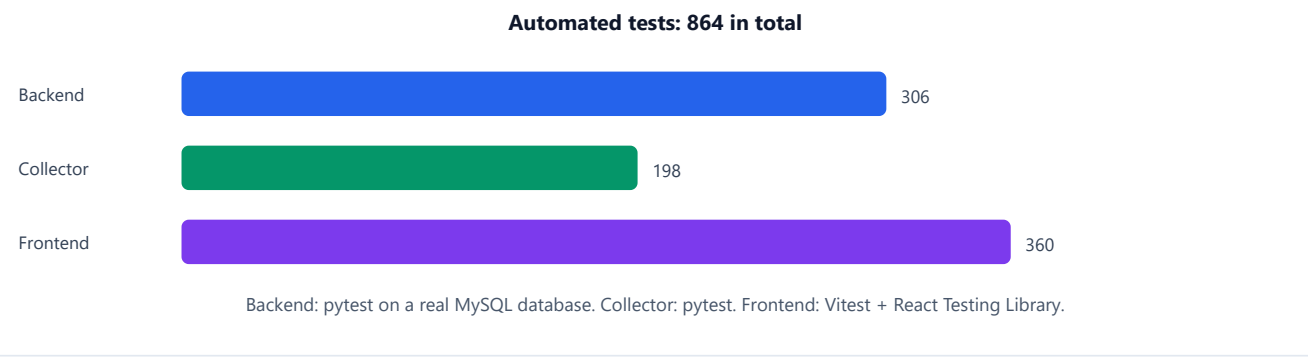



**Figure 11.** Test counts at the end of the project.

### Backend tests

Run against real MySQL inside a transaction that is rolled back, so the demo data is never changed. They cover login and roles, every endpoint, ingestion and the 30-second rule (including races and rollbacks), foreign keys, and security hardening.

### Collector tests

The Windows connection table is replaced by hand-built bytes, so tests never depend on real traffic. They cover parsing, new-versus-seen detection, domain choice, retries, bad data and Ctrl+C.

### Frontend tests

A fake `fetch` stands in for the backend. Tests cover login, route guards, every page, loading, empty, error and not-found states, and fail on any unexpected network call or console warning.

In addition, the frontend passes lint (oxlint) and builds for production without errors. Code size at the end: about 2,300 lines of backend application code, 1,000 of collector code and 3,950 of frontend code, plus roughly 6,600 lines of tests and 1,800 of SQL.

# 11 Running the project

## 11.1 Requirements

MySQL 8 or newer, Python 3.11 or newer, Node.js 20.19 or newer. The real local collector needs Windows.

## 11.2 One-time setup

```
# 1. create and seed the database
mysql -u root -p < database/schema.sql
mysql -u root -p < database/seed.sql

# 2. backend: virtual environment, packages, settings
cd backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env        # then set DATABASE_PASSWORD and a random JWT_SECRET (32+ characters)

# 3. frontend packages
cd ../frontend
npm install

# 4. collector packages
cd ../collector
pip install -r requirements.txt
```

## 11.3 Every time: three terminals

```
# terminal 1: backend
cd backend
.venv\Scripts\python.exe -m uvicorn app.main:app --port 8000

# terminal 2: frontend  (then open http://localhost:5173)
cd frontend
npm.cmd run dev -- --host

# terminal 3: collector
cd collector
$env:COLLECTOR_API_KEY="<a key from seed.sql, or one made by create_collector_key.py>"
python collector.py                      # simulator (demo data)
python collector.py local --interval 2    # real TCP connections of this PC
```

Sign in with one of the demo accounts listed at the top of `database/seed.sql` (one ADMIN, one ANALYST) and change those passwords before any real use.

## 11.4 Handy commands

| COMMAND | WHAT IT DOES |
| --- | --- |
| `python collector.py check` | Checks that the backend is reachable and the API key is accepted (stores nothing) |
| `python collector.py demo-dedup` | Shows the 30-second rule step by step |
| `python collector.py local-dns` | Reports the site names this PC looks up |
| `python create_collector_key.py --node-code HOME-PC --node-name "My Computer"` | Creates a node and a collector key (shown once) |
| `alembic upgrade head` | Applies database migrations (run in `backend/`) |
| `python -m pytest` | Runs the backend or collector tests |
| `npm test · npm run lint · npm run build` | Frontend tests, lint and production build |

# 12 Limitations and future work

## 12.1 Known limitations

- **The real collector sees one computer, TCP only.** Phones, other laptops and router traffic are invisible to it, and UDP/QUIC visits (common for large video and search sites) are not captured. Browsers that use their own DNS can leave destinations without a name.
- **No login rate limiting.** Repeated password guesses are only slowed by bcrypt itself; a limiter should be added at the web server in a real deployment.
- **No audit log** of who did what (only unhandled errors are logged).
- **Tokens cannot be revoked early** except by disabling the user, and they live in browser storage, so a Content-Security-Policy should be added at the web server in production.
- **API keys are created with a script** (or in the database); there is no screen for managing them.
- **Times are naive local server time**, with no time-zone conversion.
- **Simulated and real events look the same** in the history; only the node tells them apart.

## 12.2 Ideas for the future

- A DNS-based collector (for example reading a DNS server's log on a home network the owner controls) to see names from many devices without packet capture.
- Authorized gateway observation for UDP and QUIC metadata, kept strictly to headers and never contents.
- Alerts for unusual behavior (for example a burst of connections) as a separate, clearly scoped feature.
- A screen for managing collector keys, an audit log, login rate limiting, HTTPS deployment, and a marker column for simulated events.

## 13 Glossary

| TERM | MEANING |
|---|---|
| Node | An authorized observation point (a lab, the library, "this PC"). |
| Device | A machine seen by a node, identified by a logical identifier (not its IP). |
| Destination | A place that was reached: an IP address and, when known, a domain name. |
| Event / history row | One stored record: a device reached a destination at a time. |
| Collector | The program that observes (or simulates) activity and reports it to the backend. |
| Metadata | Facts about a connection (who, where, when, ports), not its contents. |
| REST API | A set of web addresses that accept and return JSON, used by the frontend and the collector. |
| JWT | JSON Web Token: a signed, expiring proof of login sent with each request. |
| bcrypt | A deliberately slow password-hashing function. |
| ORM / SQLAlchemy | A library that maps tables to Python classes and builds safe SQL. |
| Foreign key | A column that must match a row in another table, so links cannot break. |
| Index | A sorted lookup structure that makes searches faster. |
| Transaction | A group of changes that either all happen or none do. |
| Normalization (3NF) | Storing each fact once so data stays consistent. |
| Migration | A controlled script that changes an existing database structure. |
| TCP / UDP / QUIC | Network protocols. TCP is connection based; UDP is not; QUIC (used by HTTP/3) runs on UDP. |
| DNS | The system that turns a name such as github.com into an IP address. |